



BLOCSOC
IITD

BC106

ETHEREUM &
DE-FI

REVIEW OF BITCOIN



BLOCSOC
IITD

WHY BITCOIN IS NOT ENOUGH

- After 2009, once Bitcoin's decentralized consensus was developed a number of alternative applications rapidly began to emerge.

Thus, in general, there are two approaches toward building a consensus protocol:

1. BUILDING AN INDEPENDENT NETWORK

2. BUILDING A PROTOCOL COIN

PROBLEM WITH INDEPENDENCE:

- 1. INDIVIDUAL IMPLEMENTATION NEEDS TO BOOTSTRAP AN INDEPENDENT BLOCKCHAIN**
- 2. BUILDING AND TESTING ALL OF THE NECESSARY STATE TRANSITION AND NETWORKING CODE.**

- On the other hand blockchain-based meta-protocols, cannot force the blockchain to exclude transactions that are not valid for the meta-protocol but valid in the blockchain's protocol.
- Hence, a fully secure meta-protocol implementation would need to backward scan all the way to the beginning of the Bitcoin blockchain to determine whether or not certain transactions are valid.
- Thus, for these use cases to emerge easily there needs to be some inherent support in the existing protocol to facilitate the making of meta-protocols on top of it.

HOWEVER, THE
IMPLEMENTATION
IMPORTANT CONSIDERATIONS

In theoretical terms there are computations that can not be done in this language. No loops in it. This was done to avoid it being annoying to use.

There is no fine grain control. If you can be withdrawn, you either spend the whole thing. This also makes these scripts "one off", once someone interacts and spends the utxo the script is gone.

As a UT
ve

makes it
ols.

they do not have
(o, etc) present in

ETHEREUM

How to solve the “problems” with bitcoin.

ETHEREUM

According to the ethereum whitepaper,

The intent of Ethereum is to create an alternative protocol for building decentralized applications, providing a different set of tradeoffs that we believe will be very useful for a large class of decentralized applications.

Ethereum does this by building what is essentially the ultimate abstract foundational layer: a blockchain with a built-in Turing-complete programming language, allowing anyone to write smart contracts and decentralized applications where they can create their own arbitrary rules for ownership, transaction formats and state transition functions.

ETHEREUM PROTOCOL

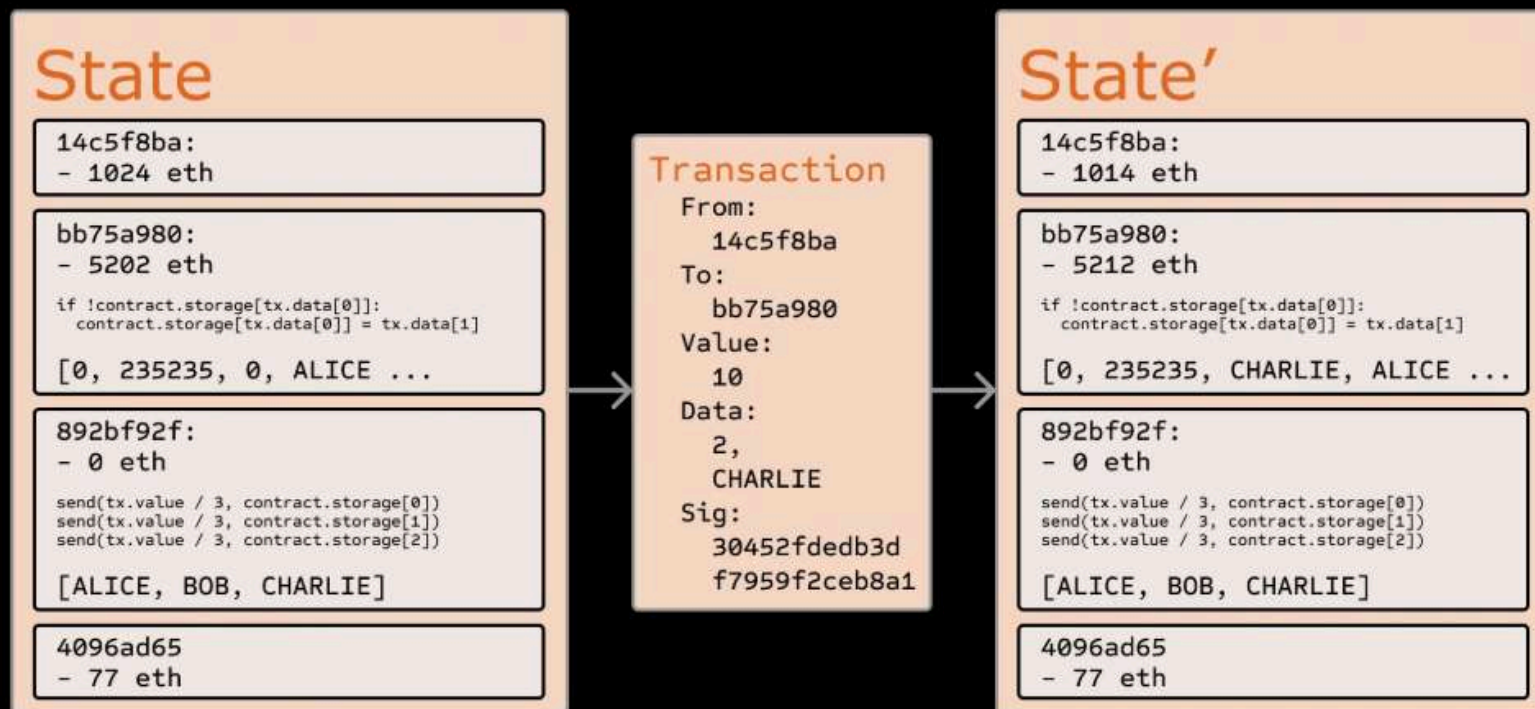
In Ethereum, the state is made up of objects called "accounts", with each account having a 20-byte address and state transitions being direct transfers of value and information between accounts. An Ethereum account contains four fields:

- The nonce, a counter used to make sure each transaction can only be processed once
- The account's current ether balance
- The account's contract code, if present
- The account's storage (empty by default)

"Ether" is the main internal crypto-fuel of Ethereum, and is used to pay transaction fees. In general, there are two types of accounts: externally owned accounts, controlled by private keys, and contract accounts, controlled by their contract code.

ETHEREUM PROTOCOL

The main way ethereum works is by using transactions to transition the state (the state consists of all accounts and smart contracts),



ETHEREUM PROTOCOL

The term "transaction" is used in Ethereum to refer to the signed data package that stores a message to be sent from an externally owned account.

Transactions contain:

- The recipient of the message
- A signature identifying the sender
- The amount of ether to transfer from the sender to the recipient
- An optional data field
- A STARTGAS value, representing the maximum number of computational steps the transaction execution is allowed to take
- A GASPRICE value, representing the fee the sender pays per computational step

ETHEREUM PROTOCOL

A transaction sent by a contract is called as a message. A message contains:

- The sender of the message (implicit)
- The recipient of the message
- The amount of ether to transfer alongside the message
- An optional data field
- A STARTGAS value

MAJOR DIFFERENCES

One big difference between Ethereum and bitcoin blocks is that ethereum also stores a merkle tree of the state in the blocks. Unlike bitcoin where the state (consisting of the UTXOs) is stored separately from the blocks in the blockchain.

The approach may seem highly inefficient at first glance, storing the entire state with each block but it is comparable to that of bitcoin.

The reason is after every block only a small part of the tree needs to be changed. Thus, in general, between two adjacent blocks the vast majority of the tree should be the same. So, if a subtree remains unchanged we can store only its hash instead of the whole subtree.

PROOF OF STAKE

Initially ethereum worked just like bitcoin, it used Proof of work (i.e. hashing power decided your share in the consensus). But, later it switched to proof of stake.

Proof of stake works on economic incentive rather than compute power. Validators have to “stake” some ETH. Each cycle a proposer is selected from the pool with probability in proportion to the staked amount.

The proposer then proposes a block which is sent to others for validation, if the block is found to be wrong some amount is “slashed” from the stake.

This is why ethereum is sometimes called “better for the environment” as it uses less overall compute power for consensus.

SMART CONTRACTS

A transaction sent to an externally owned account just increases the balance of that account. But, a transaction sent to a smart contract also causes its code to execute. This code fully describes the behaviour of the smart contract.

Code execution consumes some gas per calculation. If the gas runs out mid execution no changes are made to the state except for removing the gas amount from the sender's account.

SMART CONTRACTS

A smart contract's state also contains a long term storage. The memory used during the execution exist only during that specific run, but if you need some data to persist between runs it must be stored to the smart contract's storage.

The operations have access to three types of space in which to store data:

- The stack, a last-in-first-out container to which values can be pushed and popped
- Memory, an infinitely expandable byte array
- The contract's long-term storage, a key/value store. Unlike stack and memory, which reset after computation ends, storage persists for the long term.

EVM

Although almost every time you would need to write contract code in high level languages like solidity, the actual code of a smart contract as stored on-chain is in EVM bytecode. The solidity codes are compiled into this bytecode for execution.

EVM stands for ethereum virtual machine. You can think of virtual machines as kind of fake computers. There is no actual hardware instead it is emulated on top of some other hardware.

EVM bytecode is the machine code of this VM. And much like you can't really understand machine code of your programs, this bytecode also won't make much sense to developers. And is only meant for the EVM to execute.


```
contract AegisDeployHelper is Ownable {  
    /// @notice All deployed contract addresses  
    struct AegisAddresses {  
        address baseToken;  
        address quoteToken;  
        address oracleManager;  
        address oracleAggregator;  
        address core;  
    }  
  
    /// @notice Current deployed addresses  
    AegisAddresses public addresses;  
  
    /// @notice Pre-registered oracle addresses for easy testing  
    address[] public testOracles;  
  
    /// @notice Default test oracle names  
    mapping(address => string) public testOracleNames;  
  
    event AddressesUpdated(  
        address baseToken,  
        address quoteToken,  
        address oracleManager,  
        address oracleAggregator,  
        address core
```

SOLIDITY VS EVM BYTECODE

```
0x608060405234801561001057600080fd5b506004361061025e57  
eb60c146105fa578063f240f7c314610602578063f2fde38b14610  
e1f1c4a7146105cb57600080fd5b8063c41d24471161010a578063  
461042457806396f277b7146103d9578063aaf5eb681461047f578  
29146103d15780637d6eb017146103d95780638367a4b4146103e1  
33eef146103a6578063715018a6146103af57600080fd5b80632ba  
80fd5b806303016bf9146102635780630a763da11461029c578063  
05460ff1690565b60405161029391906130ed565b6040518091039  
5481565b6102cb6102ec3660046130aa565b610861565b6102a560  
38401546004909401546001600160a01b039093169391929091906  
565b6102cb6103a136600461312d565b610b41565b6102a5600254  
2565b60066020526000908152604090205481565b6102cb6110275
```

WHERE EXACTLY IS THE CODE RUNNING?

The results of code execution are a part of the state transition due to a transaction to the smart contract. So, to check the validity of a transition, a validator needs to execute the code.

Hence the code in a smart contract is executed everytime someone tries to verify the validity of a block. Hence, the code is being executed on every validator node.

APPLICATIONS OF ETHEREUM

TOKEN STANDARDS

As the Ethereum scripting language is so versatile there are endless ways in which one may decide to make their own currency / tokens, To allow easy interoperability and ease of use between such different tokens, some standards are set relating to what functions should be present in your smart contract, these functions define how your token behave and interact on the Ethereum blockchain.

There are a multitude of such standards, the most widely-adopted ones include,

- **ERC-20**: For fungible (dividable) tokens. This specifies transfer and amount based on “balance”.
- **ERC-721**: For non-fungible tokens (NFTs). This specifies transfer based on tokens, each token is unique and indivisible.

ERC-20 For fungible tokens

METHODS & EVENTS

```
1  function name() public view returns (string)
2  function symbol() public view returns (string)
3  function decimals() public view returns (uint8)
4  function totalSupply() public view returns (uint256)
5  function balanceOf(address _owner) public view returns (uint256 balance)
6  function transfer(address _to, uint256 _value) public returns (bool success)
7  function transferFrom(address _from, address _to, uint256 _value) public returns
8  function approve(address _spender, uint256 _value) public returns (bool success)
9  function allowance(address _owner, address _spender) public view returns (uint256
```

```
1  event Transfer(address indexed _from, address indexed _to, uint256 _value)
2  event Approval(address indexed _owner, address indexed _spender, uint256 _value)
```

ERC-721 For non-fungible tokens

METHODS & EVENTS

```
function balanceOf(address _owner) external view returns (uint256);
function ownerOf(uint256 _tokenId) external view returns (address);
function safeTransferFrom(address _from, address _to, uint256 _tokenId, bytes data) external payable;
function safeTransferFrom(address _from, address _to, uint256 _tokenId) external payable;
function transferFrom(address _from, address _to, uint256 _tokenId) external payable;
function approve(address _approved, uint256 _tokenId) external payable;
function setApprovalForAll(address _operator, bool _approved) external;
function getApproved(uint256 _tokenId) external view returns (address);
function isApprovedForAll(address _owner, address _operator) external view returns (bool);
```

```
event Transfer(address indexed _from, address indexed _to, uint256 indexed _tokenId);
event Approval(address indexed _owner, address indexed _approved, uint256 indexed _tokenId);
event ApprovalForAll(address indexed _owner, address indexed _operator, bool _approved);
```